



ESCUELA DE
INGENIERÍA EN CIENCIAS Y SISTEMAS
FACULTAD DE INGENIERÍA
UNIVERSIDAD DE SAN CARLOS DE GUATEMALA



Día, Fecha:

Martes, 14/04/2026

Hora de inicio:

17:20

INTRODUCCIÓN A LA PROGRAMACIÓN Y COMPUTACIÓN 2

LUIS ANTONIO CASTILLO JAVIER

Unidad 7:

JSON en C# y ASP.NET Core

JavaScript Object Notation con System.Text.Json & Newtonsoft.Json

Escuela de Ingeniería de Ciencias y Sistemas
Facultad de Ingeniería – Universidad de San Carlos de Guatemala

Anuncios Importantes



Anuncio 1



Anuncio 2



Anuncio 3



Anuncio 4



Anuncio 5

COMPETENCIA(S) QUE DESARROLLAREMOS



**DESARROLLAR
APLICACIONES WEB
USANDO JSON**



Agenda

Temas de la sesión

01

¿Qué es JSON?

Concepto y uso en C#

02

JSON vs XML en .NET

Comparativa práctica

03

Tipos de datos JSON

Serialización / Deserialización

04

Internet & HTTP

HttpClient en C#

05

Códigos de respuesta HTTP

HttpStatusCode enum en .NET

06

Razor Pages + Web API

Integración cliente-servidor

Competencias que Desarrollaremos

Aprendizajes esperados



Análisis de Respuestas Web

Analiza respuestas de peticiones web en C# mediante HttpClient y las librerías System.Text.Json / Newtonsoft.Json para obtener datos seguros desde APIs externas.



Comunicación Cliente-Servidor

Implementa comunicación cliente-servidor con Razor Pages como frontend y ASP.NET Core Web API como backend, usando HTTP/HTTPS para APIs seguras y eficientes.

¿Qué es JSON?

JavaScript Object Notation en el ecosistema .NET

Definición

JSON (JavaScript Object Notation) es un formato de texto ligero para el acceso, almacenamiento e intercambio de datos. Es la alternativa más popular a XML y el estándar de facto para APIs REST.

En C# / .NET

System.Text.Json

Librería nativa de .NET 6/7/8. Alta performance, ideal para ASP.NET Core Web API.

Newtonsoft.Json

El paquete NuGet más popular. Muy flexible y compatible con proyectos legacy.

```
// C# - Objeto y serialización JSON

public class Producto
{
    public int Id { get; set; }
    public string Nombre { get; set; }
    public decimal Precio { get; set; }
}

// Serializar
var json = JsonSerializer
    .Serialize(producto);

// Deserializar
var obj = JsonSerializer
    .Deserialize<Producto>(json);
```

JSON en ASP.NET Core Web API

¿Por qué es fundamental?



Ligero y rápido

JSON tiene payloads más pequeños que XML, reduciendo la latencia en las respuestas de tu Web API.



Serialización automática

ASP.NET Core serializa/deserializa objetos C# a JSON automáticamente con System.Text.Json sin configuración extra.



Universal

Compatible con Razor Pages, JavaScript, móviles y cualquier cliente HTTP que consuma tu API.



Estructuras complejas

Representa listas, objetos anidados y jerarquías complejas con pares clave-valor fáciles de leer.

Sintaxis JSON

Reglas clave y ejemplo en C#

Reglas de la Sintaxis

- Los datos se organizan en pares clave-valor ("clave": valor).
- Los elementos se separan por comas.
- Las llaves { } agrupan objetos.
- Los corchetes [] agrupan arreglos.
- Las claves deben ser strings entre comillas dobles.
- Los valores: string, number, object, array, boolean o null.

```
// JSON producido por Web API (.NET)
{
  "id": 1,
  "nombre": "Laptop",
  "precio": 1299.99,
  "disponible": true,
  "categoria": {
    "id": 3,
    "nombre": "Electrónica"
  },
  "etiquetas": ["tech","oferta"]
}
```

JSON vs XML en .NET

Comparativa para elegir el formato correcto

JSON (System.Text.Json)

Creado en 2001 por Douglas Crockford

Estructura de pares Clave-Valor

Sintaxis compacta y fácil de leer

Parseo nativo: `JsonSerializer.Deserialize<T>()`

Soporta: string, number, object, array, bool, null

Archivos más pequeños → menor latencia

Ideal para REST APIs y Razor Pages

XML (System.Xml)

Publicado en 1998 por W3C

Estructura de árbol jerárquico

Sintaxis más detallada con tags

Requiere `XmlDocument` o `XmlSerializer`

Soporta todos los tipos JSON + fechas, espacios de nombres

Archivos más voluminosos

Mejor para documentos complejos y sistemas legacy

JSON vs XML – ¿Cuándo usar cada uno?

Guía de decisión para proyectos .NET

Usa JSON cuando...

- Construyes una ASP.NET Core Web API consumida por Razor Pages, móviles o SPAs.
- Necesitas rendimiento: payloads pequeños y serialización rápida con System.Text.Json.
- Tu equipo trabaja con datos simples: listas de productos, usuarios, pedidos.
- Quieres compatibilidad universal con cualquier cliente HTTP.

Usa XML cuando...

- Integras con sistemas legacy o servicios SOAP que exigen XML.
- Manejas documentos complejos con namespaces, schemas (XSD) o firmas digitales.
- Necesitas validación estricta de tipos de datos extendidos (fechas, binarios).
- El destino final es un transformador XSLT o un flujo de datos empresarial.

Tipos de Datos JSON en C#

Mapeo entre JSON y tipos .NET

Tipo JSON	Tipo C#	Ejemplo	Anotación JsonPropertyName
string	string	"Laptop Pro"	[JsonPropertyName]
number	int / decimal	1299.99	[JsonPropertyName]
boolean	bool	true / false	[JsonPropertyName]
null	null / nullable	null	[JsonPropertyName]
object	class / record	{ "id": 1 }	[JsonPropertyName]
array	List<T> / T[]	["a","b","c"]	[JsonPropertyName]

Serialización y Deserialización en C#

System.Text.Json vs Newtonsoft.Json

System.Text.Json (.NET nativo)

```
using System.Text.Json;

// Serializar objeto C# → JSON
var producto = new Producto
{
    Id = 1,
    Nombre =
    "Laptop",
    Precio =
    1299.99m
};
string json = JsonSerializer
    .Serialize(producto);

// Deserializar JSON → objeto C#
var obj = JsonSerializer
    .Deserialize<Producto>(json);
```

Newtonsoft.Json (NuGet)

```
using Newtonsoft.Json;

// Serializar objeto C# → JSON
string json = JsonConvert
    .SerializeObject(producto);

// Deserializar JSON → objeto C#
var obj = JsonConvert
    .DeserializeObject<Producto>(json);

// Con opciones de formato
string pretty = JsonConvert
    .SerializeObject(
        producto,
        Formatting.Indented);
```

Tipos C# serializables a JSON

Equivalencias entre C# y JSON

Tipo C#	Tipo JSON	Ejemplo JSON
string	string	"Hola mundo"
int / long	number	42 / 9876543210
float / double	number	3.14 / 2.71828
decimal	number	1299.99
bool	boolean	true / false
null / null?	null	null
List<T> / T[]	array	[1, 2, 3]
Dictionary<K,V>	object	{"key": "val"}
class / record	object	{"Id": 1}

Unidad 8:

Acceso a Datos Web con C#

HttpClient, HttpResponseMessage y consumo de APIs REST desde Razor Pages

Internet y la Web

El contexto de las aplicaciones web con ASP.NET Core

Internet es un conjunto descentralizado de redes interconectadas que utilizan la familia de protocolos TCP/IP. Sus orígenes se remontan a 1969 con ARPANET. La World Wide Web (WWW) es un servicio de consulta de hipertextos surgido en 1990 que utiliza Internet como medio.

TCP/IP

Protocolo base de Internet. .NET usa System.Net.Sockets para comunicaciones de bajo nivel.

DNS

Traducción de nombres a IPs. HttpClient lo resuelve automáticamente en .NET.

WWW

Servicio de hipertextos. Las Razor Pages sirven HTML vía Kestrel/IIS.

TLS/SSL

Seguridad en capa de transporte. ASP.NET Core lo configura con certificados X.509.

Arquitectura ASP.NET Core

 Cliente (Browser)

↕ HTTP / HTTPS

 Razor Pages (.cshtml)

↕ HttpClient / JSON

 ASP.NET Core Web API

↕ Entity Framework / DB

 Base de Datos

HTTP en C# – HttpClient

Protocolo de transferencia para Razor Pages y Web API

HTTP (Hypertext Transfer Protocol) sigue el modelo cliente-servidor. En .NET, HttpClient es la clase principal para enviar peticiones HTTP. ASP.NET Core Web API es el servidor que procesa esas peticiones y devuelve respuestas JSON.

```
// Razor Pages → consume Web API con HttpClient
// Program.cs - registrar HttpClient
builder.Services
    .AddHttpClient();

// IndexModel.cshtml.cs
public class IndexModel : PageModel
{
    private readonly HttpClient _http;

    public async Task OnGetAsync()
    {
        var resp = await _http
            .GetAsync(
                "api/productos");
        resp.EnsureSuccessStatusCode();
        Productos = await resp.Content
            .ReadFromJsonAsync<List<Producto>>();
    }
}
```

Métodos HTTP → Verbos C#

GET	GetAsync(url)
POST	PostAsJsonAsync(url, obj)
PUT	PutAsJsonAsync(url, obj)
DELETE	DeleteAsync(url)
PATCH	PatchAsJsonAsync(url, obj)

Códigos de Respuesta HTTP

HttpStatusCode en ASP.NET Core Web API

Un código HTTP es un número de tres dígitos que el servidor devuelve en respuesta a cada petición. En ASP.NET Core, los controladores devuelven códigos usando los métodos de IActionResult.

1XX	Informativos El servidor recibió y está procesando.	Continue, SwitchingProtocols
2XX	Éxito Solicitud procesada correctamente.	Ok(), Created(), NoContent()
3XX	Redirección El recurso se movió a otra URL.	RedirectToAction(), Redirect()
4XX	Error del Cliente La solicitud contiene errores.	BadRequest(), NotFound(), Unauthorized()
5XX	Error del Servidor El servidor no pudo completar la tarea.	StatusCode(500), Problem()

Códigos HTTP 2XX – Éxito

Respuestas de éxito en ASP.NET Core Web API

200 OK	Solicitud exitosa.	<pre>return Ok(productos);</pre>
201 Created	Recurso creado exitosamente (POST).	<pre>return CreatedAtAction(...);</pre>
202 Accepted	Aceptada, procesamiento en segundo plano.	<pre>return Accepted();</pre>
204 No Content	Exitosa pero sin datos en el cuerpo (DELETE/PUT).	<pre>return NoContent();</pre>
206 Partial Content	Respuesta parcial con cabecera Range.	<pre>return StatusCode(206, data);</pre>

Códigos HTTP 4XX – Error del Cliente

Manejo de errores en ASP.NET Core Web API

400 Bad Request	Sintaxis inválida o datos incorrectos.	<code>return BadRequest(ModelState);</code>
401 Unauthorized	Sin credenciales válidas (JWT no enviado).	<code>return Unauthorized();</code>
403 Forbidden	Autenticado pero sin permisos.	<code>return Forbid();</code>
404 Not Found	El recurso no existe en el servidor.	<code>return NotFound();</code>
409 Conflict	Conflicto con el estado actual del recurso.	<code>return Conflict();</code>
422 Unprocessable	Bien formado pero errores de semántica.	<code>return UnprocessableEntity();</code>
429 Too Many Req.	Rate limiting: demasiadas peticiones.	<code>return StatusCode(429);</code>

Códigos HTTP 5XX – Error del Servidor

Manejo de excepciones en ASP.NET Core

500 Internal Server Error	Excepción no manejada en el servidor.	<code>throw new Exception() / StatusCode(500)</code>
501 Not Implemented	El endpoint no está implementado.	<code>return StatusCode(501);</code>
502 Bad Gateway	Respuesta inválida del servidor upstream.	<code>Proxy/Gateway config</code>
503 Service Unavailable	Servidor caído por mantenimiento o carga.	<code>Health checks + middleware</code>
504 Gateway Timeout	El upstream no responde a tiempo.	<code>HttpClient.Timeout config</code>



Tip: Usa `app.UseExceptionHandler()` y `middleware de logging (Serilog)` para manejar errores 5XX en producción.

Versiones HTTP en .NET

Evolución y soporte en ASP.NET Core

HTTP/0.9

1991

Solo GET, sin headers. No usado en .NET moderno.

HTTP/1.0

1996

POST/HEAD, headers, status codes. Conexiones no persistentes.

HTTP/1.1

1997

Keep-Alive, pipelining. HttpClient lo soporta por defecto.

HTTP/2

2015

Multiplexación, compresión de headers. Kestrel lo activa con TLS.

HTTP/3

2022

QUIC sobre UDP, menor latencia. ASP.NET Core 7+ con UseHttps().

HTTPS en ASP.NET Core

TLS/SSL, certificados y seguridad en Web API

HTTPS es la versión segura de HTTP, encriptada con TLS/SSL. En ASP.NET Core, se configura automáticamente con Kestrel. Utiliza criptografía asimétrica: clave pública para cifrar y clave privada para descifrar.

Clave Privada

Reside en el servidor. Descifra lo que la clave pública cifró. Se configura con pfx/pem en appsettings.json.

HSTS

HTTP Strict Transport Security. ASP.NET Core lo activa con `app.UseHsts()` para forzar HTTPS.

```
// Program.cs
app.UseHttpsRedirection();
app.
UseHsts();
app.
UseAuthentication();
app.
UseAuthorization();
```

Clave Pública

Disponible en el certificado. El cliente la usa para cifrar la sesión inicial (handshake TLS).

Redirección

`app.UseHttpsRedirection()` redirige todo el tráfico HTTP al puerto HTTPS automáticamente.

Ejemplo Integrador: Razor Pages + Web API

Consumo de JSON desde el frontend al backend



ASP.NET Core Web API (Backend)

```
[ApiController]
[Route(
    "api/[controller]")]
public class
    ProductosController
        : ControllerBase
{
    [HttpGet]
    public IActionResult
    GetAll()
    {
        var lista = _service.GetAll();
        return
    Ok(lista); // 200 + JSON
    }

    [HttpGet(
        "{id}")]
    public IActionResult
    GetById(int id)
    {
        var p = _service.Get(id);
        if (p == null)
            return
    NotFound();
        return
    Ok(p);
    }
}
```



Razor Pages (Frontend)

```
// Productos.cshtml.cs
public class ProductosModel
    : PageModel
{
    public List<Producto> Items { get;set; }

    public async Task OnGetAsync()
    {
        Items = await _http
            .GetFromJsonAsync
            <List<Producto>>(
                "https://api/productos");
    }
}

// Productos.cshtml
@foreach (var p in Model.Items)
{
    <p>@p.Nombre - @p.Precio</p>
}
}
```


Conceptos Clave Aprendidos

Resumen de la sesión



JSON en C#

Formato ligero de intercambio de datos. Se serializa/deserializa con System.Text.Json o Newtonsoft.Json. Soporta string, number, bool, null, object y array.



JSON vs XML en .NET

JSON: compacto, ideal para REST APIs con ASP.NET Core.
XML: mejor para documentos complejos y sistemas legacy con SOAP/WSDL.



Internet y HTTP en C#

HTTP sigue el modelo cliente-servidor sin estado. HttpClient en .NET implementa GET, POST, PUT, DELETE para consumir APIs REST desde Razor Pages.



Códigos de Estado HTTP

HttpStatusCode enum en .NET. Los controladores devuelven Ok(), Created(), BadRequest(), NotFound(), etc. según el resultado de la operación.



HTTPS y Seguridad

ASP.NET Core activa HTTPS con Kestrel. UseHttpsRedirection() y UseHsts() garantizan comunicaciones seguras con TLS/SSL en producción.



Razor Pages + Web API

Arquitectura separada: Razor Pages (frontend en C#) consume endpoints JSON del ASP.NET Core Web API (backend) vía HttpClient.



¡GRACIAS POR SU ATENCIÓN!

DUDAS & PREGUNTAS



Recuerda que tenemos nuestro Foro Semanal donde puedes consultar cualquier duda que te surja en la semana.



Ejemplo práctico



**¡GRACIAS POR SU
ATENCIÓN!**

RECUERDA QUE TENEMOS NUESTRO FORO SEMANAL DONDE PUEDES CONSULTAR CUALQUIER DUDA
QUE TE SURJA EN LA SEMANA

Referencias

Documentación oficial

Microsoft. (2024). System.Text.Json overview. Microsoft Learn. <https://learn.microsoft.com/en-us/dotnet/standard/serialization/system-text-json/overview>

Microsoft. (2024). ASP.NET Core Web API. Microsoft Learn. <https://learn.microsoft.com/en-us/aspnet/core/web-api>

Microsoft. (2024). Razor Pages in ASP.NET Core. Microsoft Learn. <https://learn.microsoft.com/en-us/aspnet/core/razor-pages>

Códigos de estado HTTP - HTTP | MDN. (2025). MDN Web Docs. <https://developer.mozilla.org/es/docs/Web/HTTP/Reference/Status>

HTTP | MDN. (2025). MDN Web Docs. <https://developer.mozilla.org/es/docs/Web/HTTP>

Newtonsoft. (2024). Json.NET Documentation. <https://www.newtonsoft.com/json/help/html/Introduction.htm>

GeeksforGeeks. (2025). HTTP Full Form. <https://www.geeksforgeeks.org/http-full-form/>